

1) Aim: To predict credit score approval using a Naive Bayes classifier based on customer data.

ALGORITHM:

- 1) Load the credit score dataset
- 2) Handle missing values using mode
- 3) perform basic data analysis
- 4) predict credit scores.
- 5) Evaluate model performance.

code.

```
import pandas as pd
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import train_test_split

df = pd.read_csv("credit_score.csv").fillna(method="ffill")
x = pd.get_dummies(df.iloc[:, :-1])
y = df.iloc[:, -1]
xtr, xte, ytr, yte = train_test_split(x, y, test_size=0.2)
print(GaussianNB()).
```

Output.

Accuracy 0.86

predicted credit scores: [1, 0, 1, 1, 0]

Result:

The naive Bayes classifier successfully predicts credit score categories with good accuracy.

2) Aim: To find the set consistent with given training examples using the candidate Elimination

Algorithm:

- 1) Initialize specific S & general hypothesis G
- 2) For each positive example, generalize S .
- 3) For each negative example, generalize G .
- 4) Output final S & G .

code:

```
def CECd():
```

```
    S = ['?'] * (len(d[0]) - 1); G = [['?'] * len(S)]
```

```
    for x in d:
```

```
        x, l = x[:-1], x[-1]
```

```
        if l == "positive":
```

```
            S = [x[i] if S[i] in ['?', x[i]] else '?' for i in range(len(S))]
```

```
            G = [g for g in G if all (g[i] == '?' or g[i] == x[i] for i in range(len(S)))]
```

```
        else:
```

```
            return S, G.
```

```
    print(CECd(["Japan", "Honda", "Blue", "1980"])).
```

Output:

Specific Hypothesis: ['Japan', 'Honda', '?', '1980', 'Economy']

Result:-

The algorithm produces the most specific and most hypothesis

3) Aim:-

To implement Linear & Polynomial Regression & evaluate their performance.

Algorithm

- 1) Load dataset
- 2) Fit Linear
- 3) Fit Polynomial model
- 4) compare errors.

code:-

```
import numpy as np
```

```
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1); y = [2, 4, 6, 8, 10]
```

```
print (LinearRegression().fit(x, y).predict(x))
```

```
p = PolynomialFeatures(2)
```

```
print (LinearRegression().
```

Output:-

Linear Regression MSE: 0.0

Polynomial Regression MSE: 0.0

Result:-

Polynomial Regression fits the data better than Linear with Lower error.

4)

Aim:

To cluster data using the Expectation maximization

algorithm:

- 1) perform Expectation step
- 2) perform maximization step
- 3) Repeat until convergence.
- 4) output clusters.

Code:

```
from sklearn.mixture import GaussianMixture.
```

```
import numpy as np
```

```
x = np.array([[1],[2],[3],[10],[11],[12]]).
```

```
print(GaussianMixture(2).fit(x).predict(x)).
```

Output:

cluster labels [0 0 0 1 1 1]

cluster means [[2.0], [11.0]]

Result:

The EM algorithm successfully groups data into meaningful clusters based on probability distributions